UNIVERSIDAD AUTÓNOMA GABRIEL RENÉ MORENO

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

Fondo de Bikini: ampliación de una ciudad interactiva con OpenGL

PROGRAMACIÓN GRÁFICA

DOCUMENTO DEL PROYECTO

MATERIA: Programación Gráfica

DOCENTE: Ing. [nombre del docente]

CARRERA: Ingenieria en informatica

INTEGRANTES:

- QUISPE CABRERA JOSE ANDRES 222009519
- RODAS CABRERA JHOEL 221045661

Índice

1. Resumen 3
2. Instrucciones de ejecución 3
2.1 Requisitos 3
2.2 Comando exacto de la versión final 3
2.3 Pruebas automáticas 3
3. Diseño de la ciudad 4
3.1 Adaptación de suelo, límites, colisiones, cámaras y minimapa 5
3.2 Colisión con la forma real del auto 5
4. Organización del código 6
4.1 El ciclo de cada cuadro 6
4.2 Dónde quedó el proyecto base 6
5. Funciones implementadas 7
5.1 Requisitos obligatorios 7
5.2 Tema personal: Fondo de Bikini 8
5.3 Mejora opcional: tráfico autónomo 11
5.4 Mejora opcional: interfaz y presentación urbana 12
5.5 Vecindario y calles con vida 13
6. Controles 17
7. Verificación 17
8. Limitaciones 18
9. Recursos externos 18

1. Resumen

Este trabajo amplía el proyecto de clase **Ciudad y conducción por etapas** según el enunciado de la actividad. La ciudad pasa de 7 × 7 a **11 × 11 celdas** conservando el tamaño de cada celda (10 unidades), con 25 manzanas conectadas por calles. Todos los componentes que dependían del tamaño del mapa (suelo, límites, colisiones, cámaras, farolas y minimapa) se calculan ahora a partir de la matriz `MAPA`, de modo que la ampliación funciona igual que la ciudad original.

Como toque personal, la ciudad se convirtió en **Fondo de Bikini**: Bob Esponja reparte cinco Cangreburgers en su botemóvil y los personajes de la serie reaccionan cuando pasa. Además se implementaron las dos mejoras opcionales: **tráfico autónomo** con cuatro botes y una **interfaz dentro de la ventana** con panel, menú de inicio y pausa, y letreros con nombres.

Además, se mejoraron los **semáforos** (dos direcciones, las tres luces visibles y luz de color real sobre la esquina), los **faroles** y los **faros**, se mejoró la **colisión** (el auto choca como un rectángulo con su forma real, se desliza por las aceras y gira sobre el eje trasero) y la ciudad se convirtió en un **vecindario con vida**: el centro conserva los edificios originales y lo rodean barrios de casas con porche, cerca y jardín, árboles más grandes, calles con bordillos y líneas, edificios con tiendas y techos detallados.

Se conservó la base técnica exigida (**Java 17, Maven, LWJGL 3.3.3 y OpenGL 3.3**) y todo el contenido del proyecto base, reorganizado por objetos y responsabilidades (sección 4). Toda la geometría se construye con un único cubo; no se usan modelos, texturas ni bibliotecas adicionales.

2. Instrucciones de ejecución

2.1 Requisitos

- JDK 17 o superior (`java -version`). No es necesario instalar Maven: el proyecto incluye **Maven Wrapper**, que lo descarga la primera vez junto con LWJGL.
- Tarjeta gráfica y controlador compatibles con OpenGL 3.3. Bibliotecas nativas incluidas para Windows x64, macOS (Intel y Apple Silicon) y Linux (x64 y ARM64).

2.2 Comando exacto de la versión final

Desde la carpeta `CiudadConduccion`, donde está `pom.xml`:

```
# Windows (PowerShell o CMD)
.\mvnw.cmd compile exec:exec

# macOS / Linux
./mvnw compile exec:exec
```

Con Maven instalado, `mvn compile exec:exec` es equivalente. El punto de entrada es `com.graphics.Main`; también se puede ejecutar desde VS Code con **Run** sobre `Main.java`. El juego abre en el menú **FONDO DE BIKINI** y se empieza con **P**.

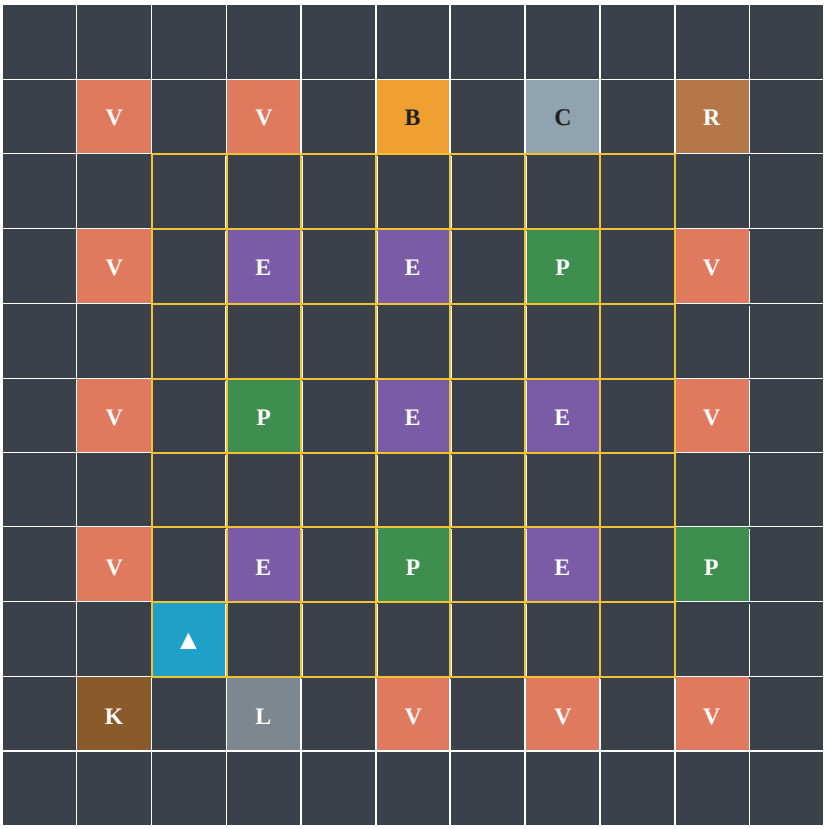
2.3 Pruebas automáticas

```
.\mvnw.cmd test
```

Se ejecutan 11 pruebas sin abrir ventanas (sección 7). Para un arranque gráfico breve se puede pasar `-Ddemo.frames=30` a la JVM del juego: la ventana se cierra sola tras 30 cuadros y cualquier error de shaders u OpenGL aparece en la consola.

3. Diseño de la ciudad

La ciudad se describe con la matriz `MAPA` de la clase `Mapa`: **0** es calle, **1** edificio, **2** parque, **3 a 7** son los lugares de la serie y **8** es una manzana residencial con cuatro casas. Las filas y columnas pares son calles, por lo que cada manzana queda rodeada por la red vial. La ciudad original de 7 × 7 se mantuvo intacta en el centro (recuadro amarillo); como el desplazamiento es de dos celdas, sus coordenadas del mundo no cambian y el auto sigue arrancando en (-30, 30).



E = edificio · P = parque · V = barrio residencial · B = piña de Bob · C = casa de Calamardo · R = roca de Patricio · K = Crustáceo Cascarudo · L = Balde de Carnada · ▲ = inicio · recuadro amarillo = ciudad original

Elemento	Cantidad	Requisito del enunciado
Tamaño del mapa	11 × 11 celdas (110 × 110 unidades)	Mínimo 11 × 11
Manzanas edificadas	16: 6 edificios altos en el centro (5 a 13 unidades, seis colores de fachada) y 10 barrios con 40 casas de uno y dos pisos	Al menos 12
Parques	4, con árboles, banco, medusas y paseantes	Al menos 4
Lugares temáticos	5 (Crustáceo, Balde, piña, moai y roca)	—

Elemento	Cantidad	Requisito del enunciado
Farolas	25 (16 en los sectores nuevos)	Al menos 9 en sectores nuevos
Destinos de entrega	5, todos en calles conectadas	Acceso por calles a todos

3.1 Adaptación de suelo, límites, colisiones, cámaras y minimapa

El enunciado aclara que ampliar solo el dibujo no cumple el requisito. Por eso los valores que estaban fijos para 7 × 7 se reemplazaron por cálculos basados en el mapa:

Componente	Antes (7 × 7)	Ahora (calculado)
Límite del mapa	`LIMITE = 35`	`LIMITE = MAPA.length * CELDA / 2` → 55
Suelo	caja de 70 × 70	caja de `2 * LIMITE`
Colisión con manzanas	círculo de radio 1.65 contra ±5	rectángulo del auto contra ±`CELDA / 2` (sección 3.2)
Cámara aérea	órbita de radio 65 y altura 55	`LIMITE * 1.85` y `LIMITE * 1.6`
Escala del minimapa	`/ 37.0` fijo en el shader	uniform `uLimite` (encuadre = `uLimite + 2`)
Farolas	9 posiciones escritas a mano	`Farola.POSICIONES`: una por manzana; el shader recibe `NUM_LUCES`

Gracias a esto, para cambiar la ciudad basta con editar `MAPA`. Los demás componentes se ajustan solos y las pruebas automáticas confirman que el resultado sigue conectado.

3.2 Colisión con la forma real del auto

El proyecto base trataba al auto como un círculo de radio 1.65 para cubrir sus esquinas. Como el auto mide 2 × 2.8, el círculo lo detenía unos 65 cm antes del bordillo por los costados, y al tocar frenaba en seco. Se reemplazó por tres mejoras, que hoy están en `Mapa` y `Auto`:

1. **Rectángulo girado (teorema del eje separador):** el auto y la manzana son dos rectángulos. No se tocan si en alguno de cuatro ejes (X, Z, el ancho y el largo del auto) sus sombras no se cruzan. Así el auto llega justo al bordillo.
2. **Deslizamiento:** si el movimiento completo choca, se intenta avanzar solo en X y luego solo en Z. Como las aceras son paralelas a los ejes, el auto se desliza a lo largo del bordillo perdiendo algo de velocidad por el roce. Si choca de frente, rebota un poco.
3. **Giro sobre el eje trasero:** el auto gira alrededor de sus ruedas traseras, como uno real, y no gira si la carrocería girada no cabe. La parte trasera ya no se mete en la acera al doblar.

Una simulación con el teclado automatizado confirmó que, al chocar en diagonal, el borde del auto queda exactamente en el bordillo (X = -25,00) y avanza deslizándose en lugar de detenerse.

4. Organización del código

El proyecto base estaba organizado en cuatro etapas encadenadas por herencia (`clase1` a `clase4`), útiles para enseñar paso a paso. Para la versión final se reorganizó todo **por objetos y responsabilidades**, como pide el enunciado: cada clase representa una cosa del programa (el auto, el mapa, un semáforo, la cámara) y sabe actualizarse o dibujarse a sí misma. Los paquetes agrupan esas responsabilidades:

Paquete	Clases	Responsabilidad
raíz	`Main`, `Juego`	Punto de entrada y director: crea los objetos, atiende las teclas y repite el ciclo de cada cuadro
`motor`	`Ventana`, `Renderizador`, `Shaders`, `Camara`, `Texto`, `Fuente`	Cómo se dibuja: ventana y teclado (GLFW), el cubo y `caja()`, el código GLSL, la cámara y las letras
`ciudad`	`Mapa`, `Ciudad`, `Calles`, `Edificio`, `Parque`, `Arbol`, `Barrio`, `Casa`, `Farola`, `Semaforo`, `LugaresBikini`	Qué hay en la ciudad: la matriz y las colisiones, y cada tipo de manzana y mobiliario
`vehiculos`	`Auto`, `Vehiculo`, `Trafico`	Conducción del jugador y tráfico autónomo
`personajes`	`Bob`, `Personajes`, `VidaMarina`	Bob, los vecinos famosos, las medusas y las burbujas
`juego`	`Iluminacion`, `Entregas`	Día/noche, faros y luces; objetivo de las entregas y cronómetro
`interfaz`	`Minimapa`, `Panel`	Vista superior en un recuadro; panel de estado y menú

4.1 El ciclo de cada cuadro

`Juego` repite unas 60 veces por segundo dos pasos. Primero **actualiza** con `deltaTime` (segundos desde el cuadro anterior): `Auto` lee el teclado y se mueve, `Entregas` revisa si llegó al destino y `Trafico` mueve los botes; en pausa `deltaTime` vale 0. Después **dibuja** en tres pasadas: la vista principal (`Camara` e `Iluminacion` envían sus datos al shader y luego se dibuja la escena), el `Minimapa` (la misma escena desde arriba en un recuadro) y el `Panel` (texto en píxeles de pantalla). Todo objeto visible tiene un método `dibujar(Renderizador r, ...)` que arma su forma con `r.caja(...)`; los que giran usan `r.origen(x, z, giro)` y `r.parte(...)` para escribir sus piezas en coordenadas locales.

4.2 Dónde quedó el proyecto base

Etapa original	Ahora está en
`clase1`: ventana, shaders, cubo, mapa y cámara orbital	`Ventana`, `Renderizador`, `Shaders`, `Mapa`, `Ciudad`, `Calles`, `Camara`

Etapa original	Ahora está en
`clase2`: auto, conducción, colisiones y cámara de seguimiento	`Auto`, `Mapa.puedeOcupar()`, `Camara`
`clase3`: día/noche, farolas, faros y shader de iluminación	`Iluminacion`, `Farola`, `Shaders`
`clase4`: ventanas, parques, pasos peatonales, semáforos, entregas y minimapa	`Edificio`, `Parque`, `Calles`, `Semaforo`, `Entregas`, `Minimapa`

La reorganización no cambió el comportamiento: las 16 capturas de este documento se generaron de nuevo con la versión por objetos y resultaron idénticas píxel a píxel a las de la versión anterior.

5. Funciones implementadas

5.1 Requisitos obligatorios

Requisito	Implementación	Dónde
Ciudad 11 × 11 con calles, edificios y parques	MAPA ampliado; la ciudad original queda en el centro	`Mapa`, `Ciudad`
Colisiones en la ciudad ampliada	Rectángulo del auto contra cada manzana, con deslizamiento por el bordillo	`Mapa`, `Auto`
Suelo, límites, colisiones, cámaras y minimapa adaptados	Todo se calcula a partir de `LIMITE` y `MAPA` (sección 3.1)	`Mapa`, `Camara`, `Minimapa`
Día y noche con N	Luz ambiental y direccional (Lambert con normales); noche azulada que permite conducir	`Iluminacion`, `Shaders`
Farolas que iluminan el entorno	25 faroles con atenuación por distancia; el vidrio emite, pero el suelo recibe luz calculada	`Farola`, `Iluminacion`
Faros con F	Dos focos que siguen al auto: haz central y relleno (`smoothstep`), atenuación y alcance limitado a 24–32 unidades; el vidrio del faro brilla al encenderlos	`Iluminacion`, `Shaders`, `Auto`
Árboles, bancos, ventanas y pasos peatonales	Generados por manzana; las ventanas se encienden de noche	`Edificio`, `Parque`, `Calles`
Semáforos animados	Dos cabezales por cruce con luces en ambas caras; ciclo de 14 s (verde 5, amarillo 2, rojo 7) controlado por tiempo; iluminan su esquina con luz de color	`Semaforo`

Requisito	Implementación	Dónde
Minimapa con M	Segundo pase ortográfico con norte arriba, posición, orientación y destino activo; se mantiene al redimensionar	`Minimapa`
Reinicio con R	Restaura auto, entregas, cronómetro y tráfico; permite repetir todo el recorrido	`Juego.reiniciar()`



Conducción nocturna por Calle Concha: faros, farolas, ventanas encendidas, destino dorado y minimapa

5.2 Tema personal: Fondo de Bikini

La entrega de paquetes se transformó en el reparto de Cangreburgers: casa de Bob → Calamardo → Patricio (las tres en Calle Concha, como en la serie) → Balde de Carnada → regreso al Crustáceo Cascarudo. Los personajes se construyen con cajas y se animan con seno y coseno del reloj.

- **Bob en el botemóvil:** el auto se reemplazó sobrescribiendo `dibujarAuto()` con `pieza()`, así que colisiones, faros y cámara siguen funcionando igual. Bob rebota más cuanto más rápido va y el motor suelta burbujas.
- **Personajes que reaccionan:** cuando el botemóvil está a menos de 14 unidades, giran para mirarlo (`atan2` con la misma convención de ángulo que el auto). Patricio duerme con burbujas de ronquido y despierta saltando; Calamardo toca el clarinete con notas flotantes y tiembla molesto si te acercas; Don Cangrejo aplaude con sus tenazas y celebra al completar las entregas; Plankton salta enojado.
- **Vida marina:** medusas que orbitan, flotan y laten sobre cada parque (brillan de noche) y 40 burbujas que suben por la ciudad. El fondo cambia a celeste de día y azul profundo de noche.



Bob Esponja al volante del botemóvil; Patricio despierto lo saluda desde la acera



Calamardo toca el clarinete frente a su casa moai mientras flotan notas musicales



Patricio dormido frente a su roca, con burbujas de ronquido y la veleta girando



Crustáceo Cascarudo con Don Cangrejo y el letrero con su nombre

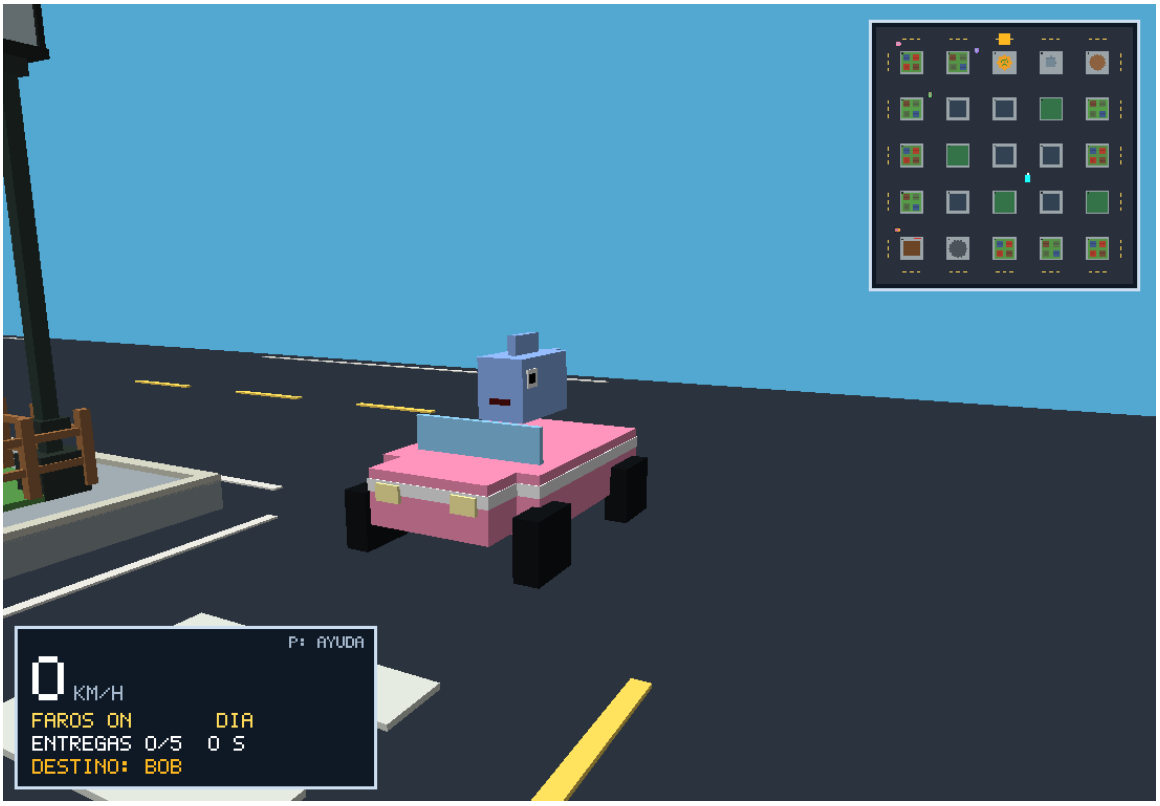


Medusas bioluminiscentes sobre un parque durante la noche

5.3 Mejora opcional: tráfico autónomo

Cuatro botes circulan sin intervención del usuario, cada uno con un pez al volante. Cada bote es un `Vehiculo` y `Trafico` coordina a los cuatro.

- Cada bote mantiene su posición, orientación y velocidad, y recorre una ruta definida como lista de cruces. Como cada tramo va por el centro de una calle, nunca atraviesa manzanas ni sale del mapa.
- Circula por su carril derecho, frena al acercarse a las esquinas y gira de forma suave.
- Se detiene si el jugador o un bote que va en su mismo sentido están delante. El jugador choca contra los botes, pero siempre puede alejarse, así que nunca queda atrapado.
- Una prueba automática simula cada ruta durante 100 segundos y verifica en cada cuadro que el bote esté sobre la calle y que complete su vuelta.



Bote del tráfico autónomo en su carril, con un pez conductor

5.4 Mejora opcional: interfaz y presentación urbana

- **Panel en la ventana** (abajo a la izquierda): velocidad en km/h, faros ON/OFF, día/noche, entregas, cronómetro y nombre del próximo destino. Se escala al redimensionar la ventana.
- **Menú de inicio y pausa** con la tecla P y la ayuda de todos los controles.
- **Texto sin bibliotecas:** cada letra es una matriz de 5 × 7 en `Fuente.java` (la misma idea que `MAPA`) y cada tramo de unos se dibuja como una caja. El vertex shader tiene un modo `uMapa = 2` que dibuja en píxeles de pantalla.
- **Mejora urbana reconocible:** letreros con los nombres «CRUSTACEO» y «CALLE CONCHA» dibujados en 3D con la misma fuente.



Menú de pausa con la ayuda de controles y el panel de estado

5.5 Vecindario y calles con vida

Los barrios, árboles y calles detalladas hacen que la ciudad se sienta habitada. Todo es decorativo: las colisiones siguen siendo las manzanas completas.

- **Barrios residenciales:** diez manzanas con cuatro casas cada una. Hay cuatro estilos (techo a dos o cuatro aguas, uno o dos pisos) con porche de columnas o balcón, contraventanas de color, alféizares, chimenea, puerta con marco, ventanas que se encienden de noche, luz de porche, flores, arbusto y buzón. Cada barrio tiene cerca blanca, de madera o celeste, con un portón frente a cada puerta.
- **Semáforos de dos direcciones:** un cabezal para cada calle del cruce, con placa amarilla, viseras y luces en ambas caras, para verlos desde cualquier dirección. Las luces apagadas se ven como vidrios oscuros de color, el amarillo dura 2 s y nunca hay dos direcciones en verde. De noche cada cabezal tiñe su esquina con su color mediante las luces de color de corto alcance del shader.
- **Faroles y faros:** faroles clásicos con base, anillo, vidrio enmarcado y techo en punta. Los faros del auto tienen un haz más largo y ancho con alcance limitado, y las luces traseras se encienden de noche y al frenar.
- **Árboles con presencia:** tronco alto y copa de tres niveles alternados a 45°, en tres tamaños, que se mecen con el agua. Están en la clase `Arbol`.
- **Calles:** bordillos claros en cada manzana, líneas blancas de borde en cada tramo y alcantarillas en los cruces.
- **Edificios con vida:** seis colores de fachada (paleta de `Mapa.colorEdificio()`), puerta con toldo de tienda, árboles de vereda, hidrante, basurero, banco, tanque de agua, aire acondicionado y antena con luz intermitente.



Barrio residencial: casas con porche, contraventanas, cerca de madera y semáforo de dos direcciones



Conducción nocturna junto al barrio: ventanas y luces de porche encendidas



Edificio con puerta y toldo, árboles de vereda, hidrante, basurero y banco



Techos con tanque de agua, aire acondicionado y antena; barrios y tráfico al fondo



Semáforo de dos direcciones: el cabezal rojo tiñe la pared y la acera cercanas



Farol clásico encendido junto a un barrio residencial

6. Controles

Tecla	Acción
P	Empezar / pausar y mostrar la ayuda
W / S o flechas arriba / abajo	Acelerar / frenar y retroceder
A / D o flechas izquierda / derecha	Girar mientras el auto se mueve
Espacio	Freno
C	Cámara de seguimiento / aérea
N	Día / noche
F	Encender / apagar faros
M	Mostrar / ocultar minimapa
R	Reiniciar auto, entregas, cronómetro y tráfico
ESC	Salir

Para entregar una Cangreburger hay que llegar a la marca dorada del destino y frenar a menos de 1 unidad por segundo. El título de la ventana mantiene el indicador básico del proyecto de referencia.

7. Verificación

`mvnw test` ejecuta 11 pruebas de lógica sin abrir ventanas:

4. El mapa es de al menos 11 × 11, con 12 o más edificios y 4 o más parques, y `LIMITE` coincide con su tamaño.
5. Una búsqueda en anchura por las calles confirma que desde el inicio se llega a todos los destinos.
6. Las calles nuevas y el borde son transitables y cada destino tiene espacio para el auto.
7. Edificios, parques y barrios (originales y nuevos), aceras y bordes bloquean el paso.
8. El rectángulo del auto llega a 5 cm del bordillo de costado y de frente, y ocupa más espacio en diagonal.
9. Hay al menos 9 farolas en los sectores nuevos y todas están sobre la acera.
10. Cada bote del tráfico circula 100 s sin tocar manzanas ni salir del mapa y completa su vuelta.
11. El jugador no puede meterse dentro de un bote, pero sí alejarse.
12. Cada carácter de la fuente tiene 7 filas de 5 cuadritos.
13. El ciclo de los semáforos dura 14 s (verde 5, amarillo 2, rojo 7) y nunca deja dos direcciones en verde.
14. El reinicio restaura la posición, la orientación y la velocidad del auto.

Además, el juego se ejecutó con `-Ddemo.frames=30` sin errores de compilación de shaders ni de OpenGL, y se revisaron con capturas de pantalla de día, de noche, en pausa y con el tráfico en movimiento.

8. Limitaciones

- La iluminación es local y sin sombras: una farola puede iluminar a través de un edificio. Los dos faros juntos pueden dejar casi blanco a un personaje muy cercano (fórmula original del proyecto).
- Los semáforos son visuales: el tráfico no se detiene en ellos, como permite el enunciado. En los barrios quedan dentro de la cerca, porque se colocan en la esquina de cada manzana.
- Los botes solo esperan a los que van en su mismo sentido; dos botes que se cruzan en una intersección pueden atravesarse un instante.
- Las burbujas atraviesan los edificios y las medusas brillan, pero no iluminan el entorno.
- La colisión usa la manzana completa: el auto choca con el bordillo, no con cada casa, cerca o árbol.
- El shader solo rota objetos alrededor del eje Y, por eso las animaciones usan traslaciones, giros horizontales y cambios de tamaño (no hay brazos que balanceen).
- La fuente no incluye Ñ ni tildes (se escribe CAMARA, DIA).
- Con Java 21 o superior, LWJGL 3.3.3 muestra advertencias de acceso nativo en la consola; no afectan la ejecución.

9. Recursos externos

- **Proyecto base:** las clases 1 a 4, `pom.xml` y la prueba original fueron entregados por el docente del curso.
- **Bibliotecas:** LWJGL 3.3.3 (GLFW y OpenGL) y JUnit 3.8.1, declaradas en `pom.xml` y descargadas por Maven.
- No se usaron fragmentos de código, modelos 3D, texturas, imágenes ni fuentes externas. Toda la geometría (personajes, edificios y letras) está construida con cajas en el propio código.
- Los personajes y lugares son referencias a la serie «Bob Esponja» (Nickelodeon), recreados con fines educativos.